Lecture 2 — C++ for Competitive Programming: STL, Fast I/O, Templates

> Year Curriculum · Prerequisites: Week 1 (judges, verdicts, I/O) and basic C++ syntax · Estimated time: 2 hours lecture + 3 hours practice

1. Overview

Most competitive programmers use C++ because it is fast and because its **Standard Template Library (STL)** provides tested implementations of the data structures and algorithms used in nearly every problem: dynamic arrays, balanced binary search trees, hash tables, heaps, sorting, and binary search. This lecture covers the part of C++ you need for contests:

1. A contest **template**: the file you start every solution from.
2. The core **containers** and the cost of each operation.
3. The `<algorithm>` and `<numeric>` functions you will use every week.
4. **Lambdas** and custom comparators.
5. The **pitfalls** that turn correct algorithms into TLE, WA, or RE.

You should finish able to choose a container from the operations a problem needs, and to state the complexity of your code including the STL calls inside it.

2. The contest template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()

void solve() {
    // solve one test case
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;          // uncomment for multi-test problems
    while (t--) solve();
```

```
    return 0;
}
```

- `#include <bits/stdc++.h>` is a GCC header that includes the entire standard library. It is non-standard and slows compilation slightly, but every major judge uses GCC. Some compilers, such as Apple's clang, lack it. On those, create the header yourself or include the headers you need.
- `using namespace std;` saves typing. Never name your own variables `max`, `count`, `y1`, `next`, or `prev` as globals, because they clash with library names.
- `ll` and `all(x)` are short aliases for things you type constantly. Keep macros few and obvious. A template full of clever macros becomes a source of bugs.
- Putting the work in a `solve()` function makes multi-test problems a one-line change.

3. Containers

3.1 `vector<T>`

A contiguous dynamic array, and your default container.

Operation	Cost
`a[i]`, `a.back()`	O(1)
`push_back`, `pop_back`	O(1) amortized
`insert`/`erase` at position	O(n)
`a.size()`, `a.empty()`	O(1)

`push_back` is amortized O(1) because the vector doubles its capacity when full. The total copying over n pushes is n + n/2 + n/4 + ... < 2n. If you know the final size, `a.reserve(n)` or constructing `vector<int> a(n)` avoids the reallocations.

2D grids: `vector<vector<int>> g(n, vector<int>(m, 0));`. For speed-critical work with fixed sizes, a global `static int g[2000][2000];` is fine. Globals are zero-initialized and do not live on the limited stack.

`size()` is unsigned. `for (int i = 0; i < a.size() - 1; i++)` on an empty vector computes `0 - 1` as an unsigned value, giving 18446744073709551615, and the loop runs off the end. Cast first: `(int)a.size() - 1`. The `sz(x)` macro does this.

3.2 `pair` and `tuple`

`pair<A,B>` holds two values. Pairs compare **lexicographically**: first by `.first`, then by `.second`. So `sort` on a `vector<pair<int,int>>` gives you a free tiebreak, and you can encode "sort by key, then by index" as `{key, index}`. C++17 **structured bindings** make the code

readable:

```
for (auto [value, index] : v) { ... }
auto [q, r] = make_pair(17 / 5, 17 % 5);
```

3.3 `string`

A `vector<char>` with string operations. Useful facts:

- `s[i] - '0'` converts a digit character to its value; `'a' + k` gives the k-th lowercase letter.
- `s.substr(pos, len)` **copies** `len` characters, so it costs O(len). Calling it inside a loop over all positions makes the loop $O(n^2)$.
- `s.find(t)` returns an index or `string::npos`. It is a naive search that can take $O(|s|\cdot|t|)$ in the worst case. Linear-time matching comes in Week 28.
- `to_string(x)` and `stoll(s)` convert between numbers and strings.
- `getline(cin, s)` reads a whole line including spaces.

3.4 `set`, `multiset`, `map`, `multimap`

These are **balanced binary search trees** (red-black trees in GCC). They keep keys in sorted order and support insert, erase, and find in O(log n).

The most important operations:

```
set<int> s = {1, 4, 9};
s.lower_bound(4);   // iterator to first element >= 4  -> 4
s.upper_bound(4);   // iterator to first element  > 4  -> 9
*s.begin();         // smallest
*s.rbegin();        // largest  (or *prev(s.end()))
```

Predecessor query. To find the largest element ≤ x, take `it = s.upper_bound(x)`. If `it == s.begin()`, there is no such element. Otherwise `--it` points to it. This pattern appears all year.

`multiset` erase trap. `ms.erase(5)` removes **every** copy of 5. To remove one copy, use `ms.erase(ms.find(5))`, and check that `find` did not return `end()`.

`map<K,V>` stores key/value pairs sorted by key. `m[k]` returns a reference to the value, **inserting a default value if `k` is absent**. That makes `m[k]++` the standard frequency-count idiom. It also means writing `if (m[k] > 0)` merely to test membership grows the map. Use `m.count(k)` or `m.find(k) != m.end()` to test.

Iterating a `map` visits keys in increasing order: `for (auto &[k, v] : m)`.

3.5 `unordered_set` and `unordered_map`

These are hash tables with **average** O(1) operations and no ordering. They are usually 2–5× faster than `map` for integer keys, with two caveats:

1. **Worst case is O(n) per operation.** GCC's default hash for integers is the identity function, and the bucket counts are predictable primes. On Codeforces, people deliberately submit tests that make every key land in one bucket, turning an O(n) solution into O(n^2) and causing TLE. The defence is a randomized hash such as the `SafeHash` (splitmix64) shown in Section 7.
2. Iteration order is unspecified. Never rely on it.

If performance is not critical, `map` is the safe default.

3.6 Adapters: `stack`, `queue`, `deque`, `priority_queue`

- `stack`: `push`, `top`, `pop` in O(1).
- `queue`: `push`, `front`, `pop` in O(1). Used for BFS (Week 16).
- `deque`: O(1) push and pop at both ends plus random access. Used for sliding-window maximum (Week 12) and 0-1 BFS (Week 17).
- `priority_queue<T>`: a binary heap and a **max**-heap by default. For a min-heap, declare `priority_queue<T, vector<T>, greater<T>>`. Push and pop are O(log n); `top` is O(1).

Weeks 12–13 cover these in depth. For now, just know they exist.

3.7 Choosing a container

Ask which operations the algorithm needs, and how often:

Need	Use
Indexed access, append	`vector`
Membership test only	`unordered_set` (safe hash) or `set`
Sorted order + insert/delete + "next larger" queries	`set` / `multiset`
Counting occurrences of keys	`map` / `unordered_map`
Repeatedly take the min/max	`priority_queue` (or `multiset` if you also need arbitrary deletion)
Both ends	`deque`

4. Algorithms

Everything here works on iterator ranges `[first, last)`.

- `sort(all(a))`: O(n log n) introsort. Pass `greater<>()` or a lambda for a custom order. `stable_sort` keeps equal elements in their original order.
- `reverse`, `rotate`, `fill`, `iota(all(p), 0)` (fills 0, 1, 2, ...).
- `min_element` / `max_element` return **iterators**. Dereference them for the value, or subtract `a.begin()` for the index.
- `accumulate(all(a), 0LL)`: the type of the initial value is the type of the sum. With `0` instead of `0LL` the sum is computed in `int` and can overflow.
- `count(all(a), x)`, `find(all(a), x)`: O(n).
- `unique(all(a))` moves consecutive duplicates to the end and returns the new logical end. Combined with `sort`, it removes duplicates: `sort(all(a)); a.erase(unique(all(a)), a.end());`. The same pattern compresses coordinates.
- `lower_bound(all(a), x)` / `upper_bound(all(a), x)` on a **sorted random-access** range: O(log n). The count of `x` in a sorted array is `upper_bound - lower_bound`.
- `next_permutation(all(a))` steps to the next lexicographic permutation and returns false after the last one. Week 3 uses it for complete search.
- `<numeric>` (C++17): `gcd(a, b)`, `lcm(a, b)`, `partial_sum`.

Trap: `lower_bound(s.begin(), s.end(), x)` on a `set` compiles, but it is **O(n)**, because set iterators are not random-access. Always use the member function `s.lower_bound(x)`.

5. Lambdas and comparators

A lambda is an inline anonymous function:

```
auto cmp = [](const pii &a, const pii &b) {
    if (a.second != b.second) return a.second < b.second;  // by second asc
    return a.first > b.first;                              // then first desc
};
sort(all(v), cmp);
```

The capture list `[]` controls access to surrounding variables. `[&]` captures everything by reference, which is the normal choice inside `solve()`. `[=]` captures by copy.

A comparator must define a **strict weak ordering**. In particular, `cmp(x, x)` must be false. Writing `<=` instead of `<` breaks this, and `std::sort` may then read out of bounds and crash with RE, not merely produce a wrong order. Week 4 covers comparators in detail.

A recursive lambda in C++17 needs a trick, such as passing itself as a parameter:

```
auto dfs = [&](auto &&self, int u) -> void { ... self(self, v); };
dfs(dfs, 0);
```

6. Worked examples

6.1 CSES 1621 — Distinct Numbers

Count the distinct values among n ≤ 2·10^5 integers. Sort, then `unique`. The answer is the length of the deduplicated prefix. Time O(n log n). Putting everything into a `set<int>` and printing its size also works, but tree nodes cost more memory and time than a sorted array.

6.2 CSES 1091 — Concert Tickets

There are n tickets with prices `h_i`, and m customers arriving in order. Customer j pays at most `t_j` and buys the most expensive remaining ticket with price ≤ `t_j`, or gets nothing (print −1).

We need a structure that supports "largest element ≤ x" and "delete that element", with duplicates allowed, so a `multiset<int>`. For each customer, `it = s.upper_bound(t)` gives the first ticket that is too expensive. The one before it, if any, is the answer. Erase **by iterator** to remove exactly one copy. Total O((n + m) log n).

Trace: tickets {3, 5, 5, 7, 8}, customers 4, 8, 3.

- t = 4: `upper_bound(4)` → 5. Step back → 3. Print 3, leaving {5, 5, 7, 8}.
- t = 8: `upper_bound(8)` → end. Step back → 8. Print 8, leaving {5, 5, 7}.
- t = 3: `upper_bound(3)` → 5, which is `begin()`. Print −1.

6.3 Codeforces 4C — Registration system

For each requested username, print "OK" if it is new. Otherwise print the name followed by the smallest number k ≥ 1 not yet used, which is simply the number of times the name has been seen before. Use a `map<string,int>`: take a reference `int &c = seen[name]`, print according to `c`, then increment. Each operation is O(|name| · log n).

6.4 CSES 1640 — Sum of Two Values (preview)

Find two positions with `a_i + a_j = x`. Scan left to right with a `map<int,int>` from value to index. For each `a_i`, check whether `x - a_i` has been seen; if not, record `a_i`. Time O(n log n). Week 6 solves it again with two pointers.

7. Reference implementations

7.1 Concert Tickets (CSES 1091)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    multiset<int> tickets;
    for (int i = 0; i < n; i++) {
        int h;
        cin >> h;
        tickets.insert(h);
    }
    for (int j = 0; j < m; j++) {
        int t;
        cin >> t;
        auto it = tickets.upper_bound(t);   // first price > t
        if (it == tickets.begin()) {
            cout << -1 << '\n';
        } else {
            --it;                           // largest price <= t
            cout << *it << '\n';
            tickets.erase(it);              // erase ONE copy
        }
    }
    return 0;
}
```

7.2 Sum of Two Values with a hash map and a safe hash (CSES 1640)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct SafeHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t R =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + R);
    }
```

```
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long x;
    cin >> n >> x;
    unordered_map<long long, int, SafeHash> pos;   // value -> 1-based index
    pos.reserve(2 * n);
    for (int i = 1; i <= n; i++) {
        long long a;
        cin >> a;
        auto it = pos.find(x - a);
        if (it != pos.end()) {
            cout << it->second << ' ' << i << '\n';
            return 0;
        }
        pos[a] = i;
    }
    cout << "IMPOSSIBLE\n";
    return 0;
}
```

7.3 Registration system (CF 4C)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    map<string, int> seen;
    while (n--) {
        string name;
        cin >> name;
        int &c = seen[name];
        if (c == 0) cout << "OK\n";
        else cout << name << c << '\n';
        c++;
    }
    return 0;
}
```

8. Debugging tools

- **Debug macro.** Debug output goes to `cerr` and appears only locally:

```
#ifdef LOCAL
#define dbg(x) cerr << #x << " = " << (x) << '\n'
#else
#define dbg(x)
#endif
```

Compile locally with `-DLOCAL`.

- `-D_GLIBCXX_DEBUG` makes the STL check bounds and iterator validity. An out-of-range `v[i]` aborts with a clear message instead of silently corrupting memory.
- `-fsanitize=address,undefined` catches out-of-bounds access on raw arrays, use-after-free, and signed overflow.

9. Common mistakes

1. **Unsigned `size()` arithmetic.** `v.size() - 1` on an empty vector, or `i < v.size() - k`. Cast to `int`.
2. **`accumulate` with an `int` initial value** silently overflows.
3. **`map[key]` as a membership test** inserts the key, growing the map and sometimes changing your logic, for example by counting keys with `m.size()`.
4. **`multiset::erase(value)`** erases all copies.
5. **Free `lower_bound` on a `set`/`map`** is O(n). Use the member function.
6. **Passing big containers by value**, for example `void dfs(vector<int> g, ...)`, copies them on every call. Use references.
7. **Invalidated iterators.** Erasing from a `vector` invalidates iterators at and after the erased position, and `push_back` may invalidate all of them. For `set`/`map`, only the erased iterator is invalidated, so use `it = s.erase(it)` when deleting inside a loop.
8. **`unordered_map` with the default hash** on Codeforces: anti-hash tests cause TLE.
9. **Comparators using `<=`**: undefined behavior in `sort`.
10. **Large local arrays.** `int a[1000000];` inside `main` may overflow the stack, which is often 8 MB or less. Make large arrays global or use a `vector`.

10. Practice problems

CSES

- 1621 — Distinct Numbers — https://cses.fi/problemset/task/1621
- 1091 — Concert Tickets — https://cses.fi/problemset/task/1091
- 1084 — Apartments — https://cses.fi/problemset/task/1084
- 1640 — Sum of Two Values — https://cses.fi/problemset/task/1640

Codeforces

- 4C — Registration system — https://codeforces.com/problemset/problem/4/C
- 160A — Twins — https://codeforces.com/problemset/problem/160/A
- 499B — Lecture — https://codeforces.com/problemset/problem/499/B
- 368B — Sereja and Suffixes — https://codeforces.com/problemset/problem/368/B

AtCoder

- ABC164 C — gacha — https://atcoder.jp/contests/abc164/tasks/abc164_c
- ABC085 B — Kagami Mochi — https://atcoder.jp/contests/abc085/tasks/abc085_b

Kattis

- oddmanout — https://open.kattis.com/problems/oddmanout
- securedoors — https://open.kattis.com/problems/securedoors

11. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 4 ("Data structures"): https://cses.fi/book/book.pdf
- cppreference.com: the container overview at https://en.cppreference.com/w/cpp/container and the algorithms library at https://en.cppreference.com/w/cpp/algorithm
- neal, "Blowing up unordered_map, and how to stop getting hacked on it", Codeforces blog entry 62393
- USACO Guide, Bronze "Introduction to Data Structures" and Silver "Sorting & Custom Comparators": https://usaco.guide/